CM4 : Variance et Biais

1 Introduction

Jusqu'à présent, nous avions des données sous la forme $\mathcal{D} = (X, Y)$. Nous cherchions à prédire Y, une classe (classification) ou une valeur (régression), sachant X. Pour cela, grâce à l'ensemble de nos données, nous entrainions un modèle $f_{\mathcal{D}}$, perceptron ou régression linéaire. L'objectif étant que en moyenne, pour une donnée $\vec{x}_n$, $f_{\mathcal{D}}(\vec{x}_n) = \hat{y}_n$ soit le plus proche possible de y_n.

Problème : pour une donnée $\vec{x}_n$ dont on connait déjà la valeur exacte y_n, cette prédiction est en elle même inutile, l'objectif réel est d'estimer $\hat{y}$ pour une donnée $\vec{x}$ dont on ne connait pas y. Comment savoir si notre modèle va pouvoir **généraliser** à de nouvelles données ?

2 Test set

Afin de "simuler" de nouvelles données, nous découpons notre dataset en un jeu **train** et un jeu **test**. On apprend les paramètres grâce aux données train et on mesure ses performances sur les données test. Ces dernières n'ayant jamais été utilisées lors de l'entrainement, elles représentent de fait, du point de vu du modèle, de nouvelles données.

Si des données sont représentés plusieurs fois dans notre dataset original, alors elles peuvent se retrouver à la fois dans train et dans set, ce qui contredit l'objectif du jeu test. On parle alors de **leakage** entre le training set et le test set. Imaginons par exemple que l'on souhaite développer un filtre à spams, si notre dataset contient un ensemble de mails sur les années précédentes, collecté chez plusieurs utilisateurs, il est fort propable qu'un même email de spam apparaisse plusieurs fois dans notre dataset (c'est un peu le principe du spam). Une solution ici pourrait consister à utiliser pour le jeu train les emails des années 2020-2024 (disons) et pour le jeu test, les emails de 2025. Cela représente mieux la tâche souhaitée, prédire les spams à l'avenir, en connaissant ceux du passé.

3 Overfitting - underfitting

Avec un modèle suffisament complexe par rapport à la taille de nos données d'entrainement, il est toujours possible "d'apprendre par coeur" les données, c'est l'**overfitting**. Par exemple, dans le cas de la régression, pour tout N points

on peut trouver un polynome de degré $N-1$ qui passe par ces N points. Dans le cas de la classification, pour tout N points de dimmension $D \geq N$ avec des labels $y \in \{-1, +1\}$, il est possible de trouver un hyper-plan séparant exactement ces points selon leur classe (**curse of dimensionality**). Ces comportements ne sont généralement pas souhaitables, puisque de tels modèles ont la plupart du temps (pas toujours) de faibles performances sur de nouvelles données, absentes de l'entrainement.

Pour mesurer l'overfit de notre modèle, on compare ses performances sur les données d'entrainement et sur les données tests. Plus le modèle overfit, plus l'écart de performances sera grand. La première raison de l'overfit est le manque de données. En effet, avec suffisament de données, l'overfit est généralement impossible, malheureusement la **quantité de données** disponible sera toujours une limite que l'on ne peut (entièrement) contrôler. L'overfit dépend également des **hyperparamètres** de notre modèle. Pour choisir ces hyperparamètres, on trace la **courbe d'apprentissage** en fonction de leurs valeurs.

Figure 1: Erreur en fonction de la compléxité du modèle (cas général - figure de gauche) et erreur en fonction de la profondeur de l'arbre (exemple concret - figure de droite).

Pour le moment nous n'avons vu que le perceptron et la régression linéaire, ces deux modèles étant très simples, ils n'ont pas (ou presque pas) d'hyperparamètres. Dans les séances suivantes nous allons justement voir diverses méthodes pour diminuer l'underfitting ou l'overfitting (kernels, PCA, abres de décision, preprocessing etc.).

4 Validation set

Comment choisir les meilleurs hyperparamètres ? On pourrait décider de mesurer leurs performances en utilisant seulement le train set, mais alors ce seront systématiquement les hyperparamètres augmentant la complexité du modèle qui seront sélectionnés. En effet, si on juge la performance du modèle avec le train set, apprendre par coeur les données est une bonne solution. On peut décider de mesurer leurs performances sur le test set, mais alors on risque d'apprendre les hyperparamètres qui fonctionnent le mieux sur ce test set spécifique et de ne

pas généraliser correctement. Cela revient au même problème que de mesurer la performance des paramètres du modèle sur le train set.

La solution est de découper nos données non pas en deux ensembles train et test, mais en trois : **train**, **validation** et **test**.

- Le **train set** sert à apprendre les paramètres de notre modèle, par exemple le vecteur w définissant l'hyperplan pour le perceptron.
- Le **validation set** sert à apprendre les hyperparamètres de notre modèle, par exemple la profondeur d'arbre max h_{max} pour les arbres de décision.
- Le **test set** ne s'utilise qu'**une seule fois** et sert à mesurer les performances finales de notre modèle. Pourquoi une seule fois ? Sinon on risque d'overfit sur nos hyperparamètres.

5 Cross-Validation

Le split entre validation et training set peut-être source de bruit, c'est à dire que l'on peut être "chanceux" ou "malchanceux" dans le choix arbitraire de la découpe. Pour limiter ce phénomène, on découpe notre jeu de donnée N fois et l'on mesure l'erreur du modèle sur chacune des découpes. Après avoir mélangé l'ordre des données, la découpe s'effectue de manière déterministe afin que chaque donnée apparaisse une fois dans un validation set :

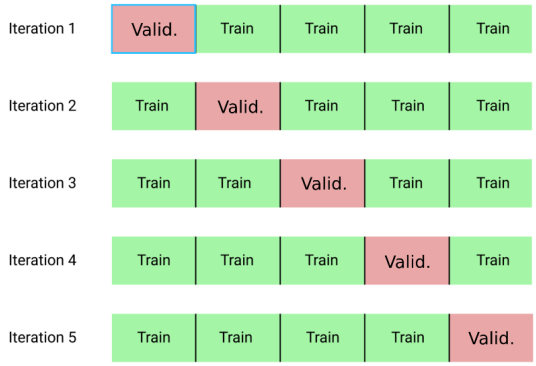

Figure 2: Cross-validation avec $N = 5$

Un bon choix est $N = 5$, cela permet à chaque itération une découpe 80% train, 20% validation. La cross-validation permettant de réduire la variance liée à la découpe arbitraire entre train/test. Est-elle est surtout utile si on a un petit ou grand nombre de données ?

6 Variance, biais, bruit

Il est possible de décrire mathématiquement l'erreur d'un modèle de la manière suivante : Erreur = Biais^2 + Variance + Bruit.

Nous n'allons pas formaliser la preuve de cette décomposition, mais simplement résumer la définition de chacun de ses termes et le lien avec ce que nous venons de voir.

- **Biais** : Le biais mesure l'erreur systématique du modèle, un biais élevé se mesure à un taux d'erreur important sur le train set et est la conséquence d'un modèle trop simple. Un biais élevé correspond à de l'underfitting.

- **Variance** : La variance mesure à quelle point l'erreur change quand on applique le modèle à des données différentes. La variance se mesure avec la différence de taux d'erreur entre le train et le test set. Une variance élevée correspond à de l'overfitting.

- **Bruit** : Le bruit correspond à l'erreur inhérente aux données, qu'il est impossible de diminuer (sans changer les données). Cela peut correspondre à des erreurs dans les données ou simplement à la nature stochastique de ce qu'elles mesurent. Imaginons par exemple que l'on souhaite prédire la réussite au bac d'un élève en fonction de sa moyenne généralle pendant l'année. Un modèle prédira par exemple la réussite pour une moyenne de 11, c'est en effet la meilleure prédiction. Ceci dit, il existera bien un élève avec 11 de moyenne et qui échouera au bac, le modèle ne pourra faire autrement que de se tromper pour celui ci. C'est le bruit.